Legacy Cipher Migration Framework for Industrial Control Systems

Project proposal — Milestone 2 (Problem Analysis)

Item	Value
Project topic	No. 8, Legacy Cipher Migration Framework for Industrial Control Systems (ICS)
Course and code	Cybersecurity / Modern Cryptography (CSET357)
Team members	Aryan Gupta, Yug Chaturvedi, Yuvraj Pathak
Instructor	Dr. Sachin
Submission format	Markdown, converted to PDF
Date	8 October 2026

> **A full report is available.** This document is the condensed version for submission and presentation. The complete analysis, with the full review of each system, extended justification and all worked detail, is in `docs/project_proposal_full.md`. Section numbers match, so a pointer such as *(see full report, B.3.1)* refers to that document. Source verification records are in `docs/references.md`.

Abstract

Legacy field devices in electricity distribution networks were installed before communication security was a design requirement, and many cannot be upgraded to run modern cryptography. A common industrial response is to place a translation gateway beside the old device: the gateway accepts the weakly protected legacy frame and forwards it under AES. This project builds a software simulation of such a gateway, implements the classical ciphers and AES named in the course problem statement, and studies which vulnerabilities persist or are introduced while a network is partly migrated. The gateway concept is established industrial practice and is not claimed here as novel; the contribution is a reproducible simulation, a set of working attacks, and measurements of what gateway-side checks do and do not catch. A feasibility prototype already runs the full pipeline end to end, recovers a legacy key by exhaustive search in under a millisecond, and delivers a forged reading to the control centre inside a cryptographically valid AES envelope.

#	Criterion	Marks	Part
1	Problem domain identification and analysis	3	Part A
2	Review of existing systems and project feasibility	3	Part B
3	Objectives and methodology of the proposed work	3	Part C
4	Relevance of algorithms and techniques	3	Part D

Every source cited was opened and read during preparation. Sources that could not be retrieved were dropped, and the omissions are recorded in B.6 and `docs/references.md`.

Part A — Problem domain identification and analysis

A.1 Domain background

NIST groups industrial control systems under operational technology, "programmable systems and devices that interact with the physical environment" [1]. In electricity distribution this means a SCADA system of three layers: field devices (RTU, PLC, IED, meter), a communication link, and a control centre. The centre "polls field devices for data at defined intervals (e.g., 5 seconds, 60 seconds) and can send new set points to field devices as required" [1]. **This project models the upward telemetry flow only.**

Three properties make these devices "legacy" *(see full report, A.1.3)*:

Property	Evidence
Very long service life	"The lifespan of an OT system can exceed 20 years", so much hardware cannot be patched [1]; SCADA equipment lifetime is given elsewhere as 20 to 50 years [3]
Severe resource limits	OT is "often resource-constrained" and may "lack desired features, including encryption capabilities" [1]; cryptographic defence on old devices is infeasible, forcing an add-on [2]
Designed before security was a requirement	Serial SCADA was "deployed decades ago with availability and personnel safety as the primary concerns, rather than security" [3]

The protocols in use are Modbus and DNP3 [3], DNP3 being IEEE Std 1815 [1], [7]; links are narrow, typically 9600 to 115200 baud with slaves transmitting only when polled [3]. Most legacy systems have "no source authentication or command verification mechanism in place" [2].

The original protection was obscurity and physical isolation [2], and that has eroded: as OT adopted IT solutions "to enable corporate business systems connectivity and remote access capabilities", the result "provides significantly less isolation for OT from the outside world than predecessor systems" [1]. A 2020 survey agrees these systems "have been exposed to the attack vectors used in the majority of cyber-attacks" while remaining "inherently much less secure" [13]. **The problem is a consequence of a change in the environment, not a fault in the original engineering.**

A.2 Problem statement

> Legacy field devices in electricity distribution networks transmit telemetry without confidentiality, integrity or source authentication, and a large installed base cannot be upgraded to run modern cryptography because of resource limits, service life and re-certification cost. The established industrial response is to place a translation gateway between the legacy device and the network, so that data leaves the gateway under AES. This arrangement moves the trust boundary rather than removing it. This project builds a simulation of such a gateway, implements the classical ciphers and AES named in the course problem statement as a graded scale of legacy protection, and determines which vulnerabilities persist or are newly introduced while the network is partly migrated.

The statement claims the migration problem is real and documented, that a gateway is the common response, and that residual vulnerabilities can be measured. It does **not** claim this project discovered the problem, that the gateway design is new, that the project offers a complete fix, or that real legacy devices use Caesar or Vigenère — they generally do not (A.6).

A.3 Root-cause analysis

#	Root cause	Effect	Why it cannot simply be fixed
R1	Protocols specified without confidentiality or authentication [2], [3]	Frames readable, alterable, injectable	Changing the protocol breaks interoperability; modification needed at protocol and device level [2]
R2	Constrained field hardware [1], [2]	Device cannot perform AES or run TLS	Infeasible on resource grounds, so protection must be an add-on [2]
R3	Service life of 20 years and beyond [1], [3]	Old and new equipment coexist for years	Capital-intensive, needs outages; phase-out will take decades [2]
R4	Re-certification and amortisation cost [4]	Even possible retrofits are rejected	High cost and long amortisation make replacement impractical; re-certification makes retrofit infeasible [4]
R5	Loss of the isolation the design assumed [1], [2]	Unanticipated threats reach the device	Connectivity is required by the business

R1–R4 explain why a gateway is used; R5 explains why it is urgent.

A.4 Threat model

The attacker is assumed to have access to the substation cabinet or local wiring — the position the literature identifies as unaddressed [4]. No cryptographic break of AES is assumed.

Position	Attacker capability	Security property lost
P1 Legacy link, device to gateway	Read, alter, inject and replay frames	Confidentiality, integrity, source authentication
P2 Inside the gateway	Observe plaintext between decryption and re-encryption; potentially read both key sets	All properties, for every device behind that gateway
P3 AES link, gateway to centre	Read and replay ciphertext	None, provided AES-GCM, sequence numbers and timestamps are used correctly
P4 Configuration	Enable passthrough or fallback	Reduces protection to the legacy level

Figure A.1 — `docs/figures/threat_model.svg` (text description in

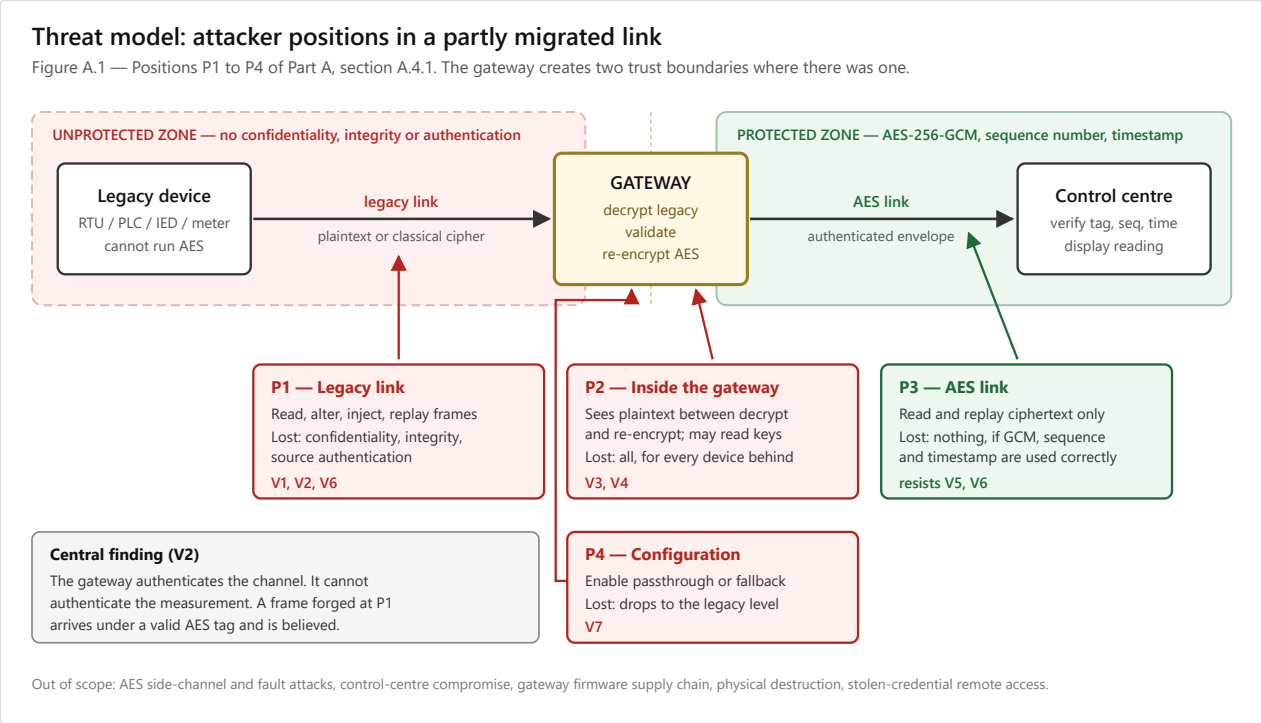


Figure A.1 — Threat model: attacker positions in a partly migrated link

`docs/figures/threat_model.md`).

The gateway creates two trust boundaries where there was one. Kaniewski et al. state the position directly: the gateway "secures only the communication on the network side but does not mitigate the absence of security features in the legacy device", so the link to the legacy device "remains vulnerable and unaddressed" [4]. That sentence is the starting point for this project.

Not modelled: AES side-channel and fault attacks, control-centre compromise, gateway firmware supply chain, physical destruction, and the stolen-credential remote-access route of the 2015 Ukraine incident. Excluded to keep the study measurable, not because they are unimportant.

A.5 Transition vulnerabilities

#	Vulnerability	Status
V1	The legacy link remains exposed	Demonstrated
V2	Authenticated false data	Demonstrated
V3	Plaintext exposure inside the gateway	By design, argued

#	Vulnerability	Status
V4	Two key sets on one device	Argued
V5	Incorrect AES mode selection	Planned
V6	Replay on the legacy link	Demonstrated
V7	Downgrade to passthrough	Planned
V8	Added latency and a single point of failure	Partly measured

V1 — AES protects only the half beyond the gateway; the device-to-gateway segment stays exposed, which is what the literature leaves open [4]. The prototype broke a 36-key additive cipher in 0.2039 ms.

V2 — The gateway cannot distinguish a genuine frame from a forged one, because the legacy protocol provides no source authentication [2], so it wraps a tampered reading in a valid AES envelope and the centre sees a correctly authenticated message. Chan and Zhou describe the consequence: false readings "could present a distorted picture of the system status to the master station, thus possibly triggering a false alarm and disruptive actions" [2]. **This is the most serious finding: cryptography on the protected half performs exactly as specified while the data it protects is false. Authentication of the channel is not authentication of the measurement.**

V3 — Translation requires decryption, so cleartext necessarily exists inside the gateway, which typically serves several devices. In the 2015 Ukraine incident attackers "rendered Serial-to-Ethernet devices at substations inoperable by corrupting their firmware" [5] — this device class is reachable and targeted.

V4 — The gateway holds a legacy key per device plus the AES uplink key, and field sites are visited rarely, making rotation hard. The project records the exposure rather than solving key distribution (A.6).

V5 — Choosing AES is not sufficient: ECB reveals repeating structure and telemetry is repetitive, while CBC without separate authentication gives no integrity.

V6 — Legacy frames carry no freshness information. The prototype shows the asymmetry: a replayed AES envelope is rejected on its sequence number; a replayed legacy frame is accepted and forwarded.

V7 — Bypass modes exist for commissioning. If triggered or left enabled after maintenance, protection returns to the legacy level while the system appears operational.

V8 — Encryption "adds latency and may not be suitable for all OT devices" [1], and latency dominates retrofit design for time-critical serial links [3]. A gateway is also a single point whose failure removes visibility of every device behind it.

A.6 Justification, scope and limits

Documented incident. On 23 December 2015, three Ukrainian distribution companies suffered outages affecting approximately 225,000 customers, assessed as caused by external attackers [5]. Breakers were operated remotely by "multiple external humans using either existing remote administration tools at the operating system level or remote industrial control system (ICS) client software via virtual private network (VPN) connections", with credentials believed acquired beforehand [5]. **Two honest qualifications:** that route was stolen credentials, not legacy-link interception, so a gateway would not have prevented it; and CISA states BlackEnergy was present but its role is not established [5]. It is cited to show field-equipment manipulation has large real consequences — not as evidence for the attack simulated here. The 2001 Australian sewage incident reported in [3] is more comparable. *(Stuxnet is deliberately not cited — see full report, A.6.1, and T4.)*

Standards that exist in response *(see full report, A.6.2)*: NIST SP 800-82 Rev. 3, whose compensating-control concept justifies gateway-side validation [1]; IEC TS 62351-1:2007 [6]; IEEE Std 1815 (DNP3) [1], [7]; and IEEE Std 1711.1 (SSPP) and 1711.2-2019 (SSCP) [1], [8].

Item	Content
In scope	Software simulation on one machine; upward telemetry only; one gateway; several simulated devices; the named ciphers and AES; attacks on the legacy link; gateway-side validation and its measurement
Out of scope	Real hardware; genuine Modbus or DNP3 framing; downward control commands; key distribution infrastructure; post-quantum cryptography; the stolen-credential route
Assumptions	Gateway adjacent to the device; AES key provisioned securely in advance; clocks synchronised within a 30-second window; frame layout is public, as a published protocol would be

Stated limitation, carried throughout. Real legacy devices overwhelmingly transmit plaintext or proprietary encodings; they do not use Caesar, Affine, Playfair, Vigenère or Rail Fence. These ciphers are a deliberate teaching device giving a graded scale of

weak protection from a key space of one upward. Where the literature describes the legacy link it describes it as unprotected [4] — the zero point of that scale. Any conclusion depending on a specific classical cipher is a statement about that cipher, not about field practice.

Part B — Review of existing systems and project feasibility

B.1 Review scope

Eleven systems were reviewed across three categories — five standards, four research works and two commercial products — each assessed for what it does, how, and what it does not do, then compared on identical criteria in B.2. Ten appear as rows in that table; IEEE 1711.1 appears as a note, because no verified source supports any property beyond its title. Two surveys [2], [13] are used as corroborating evidence rather than reviewed as systems. Only sources that were opened are cited; unverified cells are marked, not guessed. *(Full descriptions of every system: see full report, B.2 to B.4.)*

Four findings carry into the rest of this document. **IEC 62351's scope excludes Modbus** — the series targets IEC TC 57 protocols [6] — so a utility running Modbus cannot simply adopt it. **IEEE 1711.1 addresses substation serial links**, this project's exact scenario, which shows the Part A problem is recognised at standards level [1], [8]. **YASIR quantifies the latency constraint**: at 9600 baud a byte takes about 1 ms, so two bump-in-the-wire modules buffering a 20-byte message add 40 ms from buffering alone, against a 10 ms requirement for IED protection messages [3]. And **in-device AES is the right answer where it is possible**: Nguyen et al. achieve 41.08 µs encryption on Siemens S7-1200 and Rockwell CompactLogix PLCs with full interoperability [12] — but it requires a PLC capable of running AES, which the constrained devices defining this problem are not [1], [2].

One observation shapes Part D: Tsang and Smith note that "retrofitting data privacy to the communications in serial-based SCADA systems, even the time-critical ones, is a relatively trivial task; the real challenge lies in retrofitting data authenticity and freshness in a timely manner" [3]. Hallmark reaches the same conclusion from the engineering side, making authentication primary in SSCP and encryption optional [10].

B.2 Comparison of reviewed systems

"Legacy link protected?" asks specifically whether the segment between the legacy device and the protecting element is covered.

System	Approach	Legacy device modified?	Confidentiality	Integrity and authentication	Replay protection	Legacy link protected?	Reported latency	Limitation
NIST SP 800-82 r3 [1]	Guidance, compensating controls	No	Not a protocol	Not a protocol	Not a protocol	Not addressed as a protocol	n/a; notes encryption adds latency	Guidance only; prescribes no mechanism
IEC 62351 series [6]	Protocol security for IEC TC 57 families	Yes — changes at both protocol and device level [2]	Series covers "data and communications security"; IEC 62351-8 provides authentication and authorization rather than confidentiality [2], [6]	Yes — authentication and access policy enforcement; can avoid or minimise false command injection [2]	Not stated by the verified sources	**No — scope excludes Modbus** [6]	No figure; [2] reports computational and communication overheads as the obstacle	Challenging to deploy on legacy systems and protocols [2]
IEEE Std 1711.2-2019, SSCP (see note below) [1], [8], [10]	Protocol-independent secure serial protocol	No	**Optional** [10]	Yes — per-message authenticator; unauthenticated commands logged as errors [10]	Not stated explicitly; unique identifier per message [10]	No — protects beyond the module, not device-to-module	Not reported	Encryption optional, so adoption alone does not give confidentiality

System	Approach	Legacy device modified?	Confidentiality	Integrity and authentication	Replay protection	Legacy link protected?	Reported latency	Limitation
IEEE Std 1815, DNP3 + Secure Auth [7], [14]	Protocol extension	Yes — both endpoints must implement it [14]	Not listed among the publicly documented DNP3 features [14]	Yes — secure authentication is part of the DNP3 document set [14]	Not stated in the openly readable documentation [14]	No — endpoint-to-endpoint	Not reported	Original DNP3 deployed unsecured [9]; DNP3's documented "high data integrity" refers to layered-protocol error detection, not cryptographic protection [14]
YASIR [3]	Bump-in-the-wire, serial	No	Yes	Yes, authentication tag	Yes, sequence number	No — device-to-module segment uncovered	Transmitter ≤ 4 byte times, receiver 14; ≈ 1 ms per byte time at 9600 baud [3]	Latency grows with frame length; serial links only
Predictive YASIR [9]	Bump-in-the-wire with traffic prediction	No	Yes	Yes	Not reported	No — same architecture	15.32 ± 0.27 % latency improvement [9]	Simulation only; gain depends on predictable traffic
In-device AES-128 on PLCs [12]	In-device cryptography	**Yes — PLC program change**	Yes, AES-128	Not reported	Not reported	Not applicable — no legacy link exists	Encryption 41.08 µs; end-to-end 5–35 ms [12]	Requires a capable, programmable PLC, so unavailable on the constrained devices that define the problem [1], [2]
Compensating measures [4]	Analysis of physical, technical, operational measures	No	Partly, per measure	Partly, per measure	Not addressed	**Yes — its explicit subject**	Not reported	Evaluation framework, not an implemented system
Hallmark SEL-3025 / SEL-3045 [10]	Commercial bump-in-the-wire module and OEM card	No	Optional, per SSCP [10]	Yes [10]	Not stated explicitly; unique identifier per message [10]	No — device-to-module segment uncovered	"Minimal", no figure given → not reported [10]	Proprietary hardware purchase; legacy link remains
Thales Datacryptor SCADA [11]	Commercial bulk encryptor	No	Yes, AES [11]	Administrative access only, not per-message [11]	Not reported	No	Not reported	2005 trade announcement of a product then in development; status not verified
This project	Simulated protocol-translation gateway	No (devices simulated)	Yes, AES-256-GCM on the uplink	Yes on the uplink, GCM tag over header and payload	Yes on the AES link (sequence number, 30 s window); **no on the legacy link, by design**	**No — studied as the object of the work, not fixed**	Gateway path 10.69 µs, measured; see B.4	Simulation only; classical ciphers are a teaching stand-in; no real Modbus or DNP3 framing

Note on IEEE Std 1711.1 (SSPP). No row of its own, because no verified source supports any cell beyond its title and a row of empty cells would mislead. Its designation and subject are established [1], [8], and its existence is evidence the Part A problem is recognised at standards level.

B.3 Gap statement

Finding from the review	Position of this project
Gateway-based retrofit is established and commercially available, standardised as IEEE 1711.2 and sold as the SEL-3025 link module [8], [10]	Reproduced in simulation. No part of the architecture is claimed as new
Existing work assumes the legacy protocol carries plaintext [3], [4], [9]	This project adds classical ciphers as a graded scale of weak legacy protection, for teaching and comparison, while stating this is not field practice
The legacy link is acknowledged as the remaining weakness and left unaddressed by gateway designs [4]	This project demonstrates the consequence with working attacks and measures what simple gateway-side checks do and do not catch

The gateway is not novel. It is a software reproduction of an architecture that has existed in commercial hardware for close to two decades [10], [11] and is standardised [8]. The contribution is not a mechanism but evidence: a reproducible simulation in which the residual vulnerabilities of Part A can be exercised, measured and shown, with an honest account of the limits of the mitigations. Both the research [3] and the engineering [10] agree that wrapping telemetry in AES is the straightforward half, and establishing that the reading was true is the hard half.

B.4 Feasibility analysis

Technical. The prototype (`prototype/demo.py`, output in `docs/prototype_run.txt`) runs the whole pipeline with one command and no arguments. Measured per-stage cost, mean of 2000 runs:

Stage	Mean		Stage	Mean
Caesar encrypt / decrypt	4.16 / 3.97 µs		Parse	1.60 µs
Vigenère encrypt / decrypt	5.44 / 5.81 µs		Validate	1.97 µs
AES-256-GCM seal / open	3.15 / 3.08 µs			

```
legacy decrypt 3.97 + parse 1.60 + validate 1.97 + AES seal 3.15  =  10.69 µs gateway path
```

For context, YASIR cites a 10 ms requirement for IED protection messages and up to 0.2 s for a poll response [3]. Our measured cryptographic and parsing cost on a laptop is roughly three orders of magnitude below the tighter of those figures. **This is not a claim that the design meets that requirement.** It suggests only that cryptography is unlikely to be the bottleneck in this architecture; it is not a prediction of latency on real gateway hardware, where transmission time dominates — at 9600 baud a single byte takes about 1 ms [3], which alone exceeds our whole measured processing path.

Two further qualifications, from the run log itself: the classical ciphers are pure Python while AES runs in a compiled backend with AES-NI, "so the absolute figures compare implementations, not the algorithms themselves"; and the figures are for one machine and "are not a prediction of latency on real gateway hardware". The 10.69 µs is computation only and excludes transmission entirely.

Three results confirm the vulnerabilities are reproducible:

- **Stage 2 (V1)** — all 36 additive keys tried in 0.2039 ms by ciphertext-only exhaustive search with a plaintext-recognition test; one candidate passed, recovering key `k = 3` and the reading.
- **Stage 3 (V2)** — with validation off, an edited frame reached the centre as `290.1 V` against a true `230.1 V`: "GCM tag verified, yet the voltage is false". With validation on the gross edit was rejected ("V=2901 outside plausible range [2000, 2600]"), but a small edit to `231.1 V` was still accepted. The log's own conclusion: "Validation reduces the problem. It does not remove it."
- **Stage 4** — one flipped bit in the ciphertext and one in the header were each rejected with "InvalidTag: GCM tag check failed". The envelope is sound, which is precisely why a forged reading inside it is believed.

Verdict: feasible. Every mechanism runs end to end, and measured cost is far below the timing constraints reported in the literature.

Economic. No hardware, no licence; one pinned dependency (`cryptography`) plus `pytest` and `matplotlib`, all free, on existing laptops. **Verdict: feasible** — monetary cost zero, only team time is consumed.

Operational. One command, no arguments, labelled output naming the vulnerability at each stage, readings printed in engineering units. **Verdict: feasible** — readable output and a rehearsal path not dependent on live execution.

Schedule. This document and the prototype exist and the architecture is proven end to end, so remaining work extends working code rather than resolving design problems; the ciphers are small and independently testable, dividing naturally among three members. **Verdict: feasible, subject to dates**. The work plan in C.7 is stated in relative weeks, because the remaining milestone dates had not been issued when this document was prepared (OI4).

Legal and ethical. Entirely simulated: no real grid equipment, utility network or vendor product is touched, telemetry is synthetic, and attacks run only against the project's own components. Keys are generated at run time or read from the environment, never in source. **Verdict: feasible.**

B.5 Risk register

#	Risk	Likelihood	Impact	Mitigation
K1	Classical ciphers cannot handle numeric telemetry	High	High	One 36-character alphabet `A–Z` then `0–9`, arithmetic mod 36. Implemented and verified in the prototype
K2	Playfair padding corrupts readings	Medium	High	`X` reserved as filler and excluded from readings; parser strips every `X`. Covered by round-trip tests
K3	AES misuse — nonce reuse or a weak mode	Medium	High	GCM only, fresh random 96-bit nonce per message, library primitives, header authenticated as associated data. Verified by Stage 4
K4	Frequency analysis weak on short structured messages	Medium	Medium	Use fixed field tags for known-plaintext attacks, and exhaustive search with a recognition test where the key space is small
K5	Scope creep	Medium	Medium	The out-of-scope list in A.6 is treated as binding
K6	Demo fails on another machine	Low	High	Pinned versions, single run script with no arguments, recorded backup run. Already executed on a second machine
K7	A cited source becomes unreachable before submission	Low	Medium	Every source opened during preparation; verification status recorded in `docs/references.md`

B.6 Sources dropped

Two works named in the project brief could not be opened and are cited nowhere: the University of Messina proxy paper (HTTP 403) and the Modbus/TCP Security specification (HTTP 404; the index page is JavaScript-rendered). Consequently the proxy category rests on an analysis [4] rather than an implementation, and the claim that Modbus carries no security rests on secondary sources [1], [2], [3]. The Maroochy MITRE report was also inaccessible, so the 2001 incident is cited through [3]. See `docs/references.md`.

One further limitation of this review, stated rather than concealed. Both commercial entries are old: the Thales report dates from 2005 [11] and the Hallmark fact sheet is undated. No current vendor datasheet for a serial security gateway was obtainable during preparation, so the review establishes that the architecture *is* commercially established but says nothing about the current state of the market (OI10).

Part C — Objectives and methodology of the proposed work

C.1 Aim

To build a reproducible software simulation of a legacy-to-AES protocol translation gateway, and to use it to demonstrate and measure which security weaknesses persist, and which are newly introduced, while an industrial control network is partly migrated.

C.2 Objectives

No objective sets a numeric performance target; inventing one would also presuppose the answer to the questions in C.3. Timing references are external: 10 ms for IED protection messages and 0.2 s for a poll response [3].

#	Objective	Deliverable	Measurable success criterion
O1	Implement the six classical ciphers and the AES-256-GCM envelope	`ciphers/`, `gateway/aes_envelope.py`	`Decrypt(Encrypt(m)) = m` for the full test vector set including edge cases; invalid Affine keys rejected; altered envelope, header or tag rejected in every case
O2	Build gateway, device simulator and control centre as separate processes	`telemetry/`, `gateway/`, `control_centre/`	A reading arrives byte-identical at the centre across all six ciphers; a reason logged for every accept and reject
O3	Implement the attacks on the legacy link	`attacks/`	Each cipher's key recovered by the stated technique with time recorded; a replayed legacy frame accepted by an unprotected gateway
O4	Demonstrate and measure V1 to V8	`attacks/`, `experiments/`	Each of V1–V8 has a scripted demonstration producing the predicted outcome, or a written statement of why it is argued rather than demonstrated
O5	Evaluate gateway-side mitigations and state their limits	`gateway/validator.py`, `experiments/detection_rate.py`	Detection rate against tampered and replayed frames reported for validation off and on, with the frames that pass undetected characterised and the false-alarm rate stated

O5 deliberately sets no threshold: the prototype already showed the answer is below 100 %, so the measurement of interest is *which* frames escape.

C.2.1 Traceability

Objective	Vulnerability	Module	Experiment	Metric	Success criterion
O1	V5	`ciphers/*`, `aes_envelope.py`	E3, E2	Payload bytes; µs per primitive	Round trip exact; ECB and GCM distinguishable in E5
O2	V3, V4, V8	`device_sim.py`, `gateway.py`, `profiles.py`, `centre.py`	E2, E3	µs per stage; envelope size	End-to-end identity holds; per-stage cost reported alongside the 10 ms figure from [3] as context, not as a claim of meeting it
O3	V1	`brute_force.py`, `known_plaintext.py`, `frequency.py`, `kasiski.py`	E1	Seconds to recover each key	Key recovered for every cipher, time recorded
O3	V6	`mitm_proxy.py`	E4	Share of replays accepted	Legacy replay accepted without sequence checks; AES replay rejected
O4	V2	`mitm_proxy.py`, `validator.py`	E4	Share of tampered frames reaching the centre	Reported for both validator states; undetected frames characterised by edit size
O4	V5	`ecb_demo.py`	E5	Repeated ciphertext blocks	Repetition visible under ECB, absent under GCM
O4	V7	`gateway.py` passthrough	E4	Frames accepted in passthrough	Downgrade shown and logged
O4	V8	`timing.py`	E2	µs added per frame	Reported with its implementation caveat
O5	V2, V6	`validator.py`	E4, E6	Detection rate; false alarms	Both states reported; misses characterised; false-alarm rate stated

V3 and V4 sit under O2 because they are properties of the architecture, not behaviours that can be triggered: the gateway necessarily holds plaintext and two key sets. They are argued from the design, not demonstrated by a script.

C.3 Research questions

Six questions drive the work, each answered by a named experiment: break time per cipher and whether classical cryptanalysis works at all on a 27-character frame (Q1, E1); the share of tampered frames validation detects and which kinds pass (Q2, E4); the false-alarm cost of validation (Q3, E6); gateway processing cost against the constraints in [3] (Q4, E2); envelope overhead on a short frame (Q5, E3); and whether AES mode choice changes what leaks from repetitive telemetry (Q6, E5). *(Full wording: see full report, C.3.)*

Q1 carries a genuine uncertainty: classical cryptanalysis assumes long natural-language text, and a 27-character structured frame is neither. If frequency analysis and Kasiski perform poorly on single frames, that is itself a finding and will be reported as one.

C.4 Methodology

Simulation and controlled comparison, not invention. An architecture that already exists (Part B) is reproduced in software, instrumented so every stage can be timed and every decision logged, then attacked under two configurations — validation off and on — so the effect of the mitigation is isolated.

Figure C.2 — `docs/figures/methodology.svg`

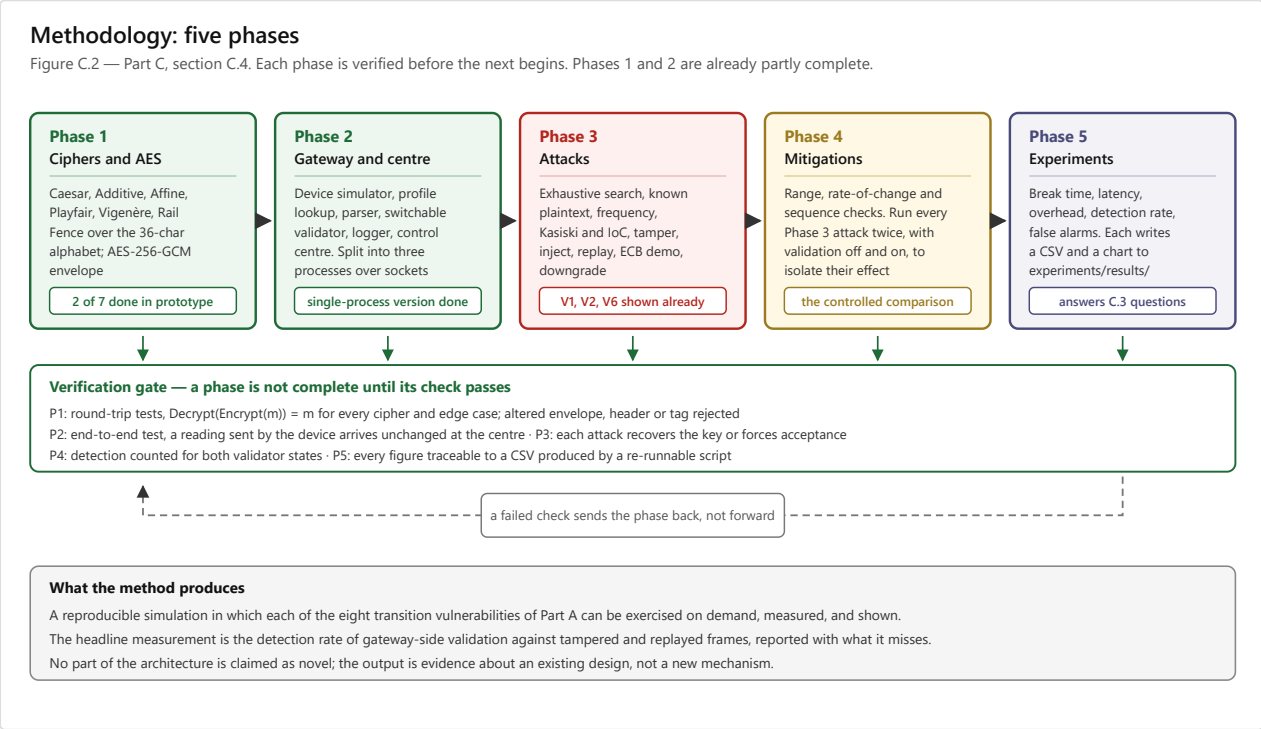


Figure C.2 — Methodology: five phases

Phase	Inputs	Activities	Outputs	How it is verified
P1 Ciphers and AES	Cipher definitions; 36-char alphabet; frame format	Implement the six ciphers from scratch; implement the AES-256-GCM envelope with header as associated data	`ciphers/`, `aes_envelope.py`, unit tests	Round-trip property for every cipher; invalid Affine keys rejected; altered envelope, header or tag rejected
P2 Gateway and centre	P1 output; device profiles	Device simulator, profile lookup, parser, switchable validator, logger, control centre; split into three processes over local TCP	`telemetry/`, `gateway/`, `control_centre/`, `run_demo.py`	End-to-end test: a reading arrives unchanged, for every cipher
P3 Attacks	P2 output	Exhaustive search, known plaintext, frequency, Kasiski and IoC, MITM proxy for tamper/inject/replay, ECB demo, downgrade trigger	`attacks/`	Each attack produces its predicted outcome

Phase	Inputs	Activities	Outputs	How it is verified
P4 Mitigations	P3 output	Range, rate-of-change and sequence checks; re-run every Phase 3 attack with validation off then on, all else constant	`validator.py` , paired result sets	Detection counted for both states over the same attack set and seed
P5 Experiments	P4 output	Run E1–E6, write CSV and charts, answer Q1–Q6	`experiments/results/`	Every figure traceable to a CSV from a re-runnable script

Already complete. The prototype discharges part of P1 (Caesar, Vigenère, AES envelope), part of P2 (reading, parser, switchable validator, control centre with tag, sequence and timestamp checks), part of P3 (exhaustive search, tamper, replay, envelope bit-flip) and the P5 timing harness. Remaining: four ciphers, per-device profiles, the three-process split, the remaining attacks, systematic paired runs, and E1 and E3–E6. (See full report, C.4.3.)

C.5 System design

Figure C.1 — `docs/figures/architecture.svg`

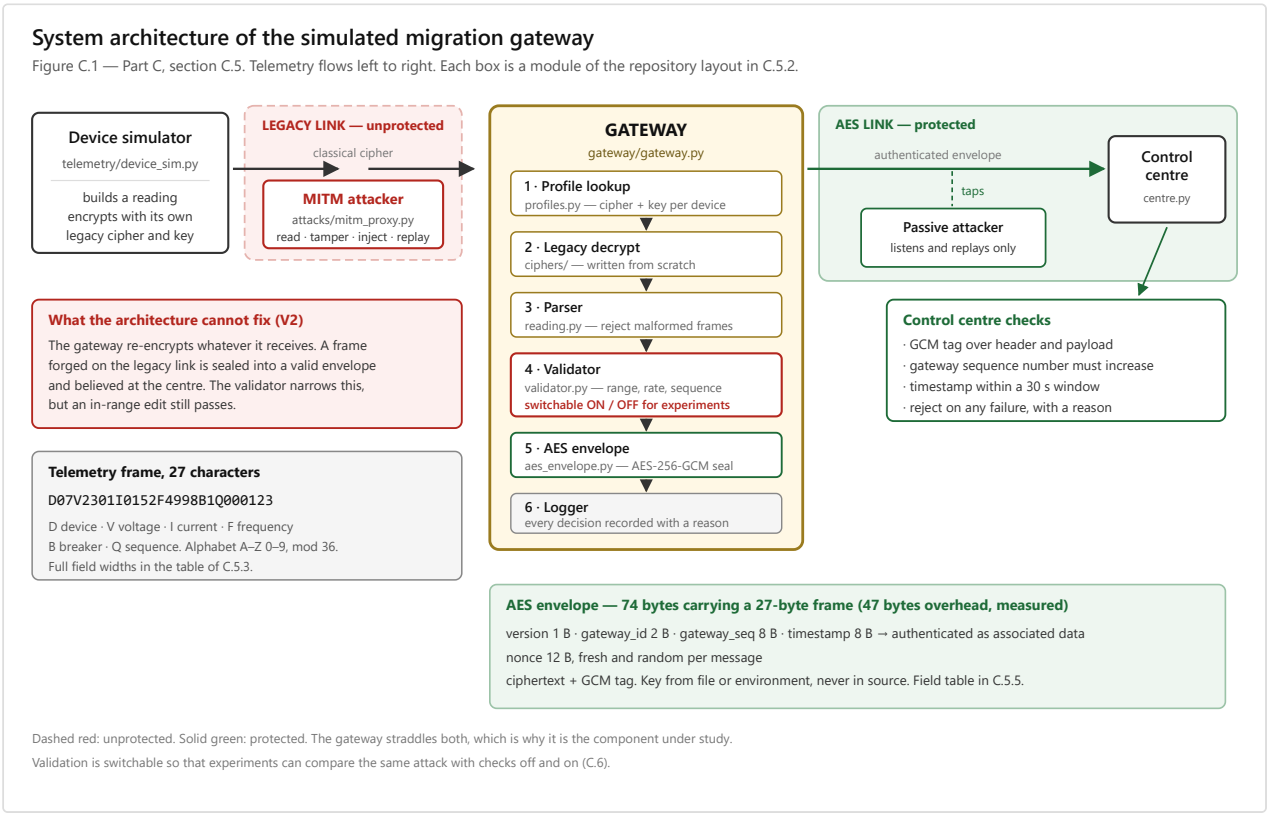


Figure C.1 — System architecture of the simulated migration gateway

Telemetry flows from the device simulator across the unprotected legacy link, through six gateway stages, across the AES link, to the control centre. A man-in-the-middle sits on the legacy link; a passive attacker taps the AES link.

Telemetry frame — fixed width, no separators, 27 characters, every character from the 36-character alphabet. Example `D07V2301I0152F4998B1Q000123`.

Tag	Field	Width	Meaning	Example
`D`	Device ID	2	Identifies the device	`07`
`V`	Voltage	4	Tenths of a volt	`2301` = 230.1 V
`I`	Current	4	Tenths of an ampere	`0152` = 15.2 A
`F`	Frequency	4	Hundredths of a hertz	`4998` = 49.98 Hz
`B`	Breaker	1	0 open, 1 closed	`1` = closed
`Q`	Sequence	6	Device counter	`000123`

`X` is reserved as the Playfair filler and never appears in a reading, so the parser can strip it unambiguously.

Gateway pipeline: receive → profile lookup (reject unknown device) → legacy decrypt → parse (reject malformed) → validate (reject with the failing check named) → build AES envelope and send → log the decision and reason. **Step 5 must be switchable at run time**, because the P4 comparison depends on running the identical attack set in both states.

AES envelope

Field	Size	Content	Authenticated
`version`	1 byte	Envelope format version	Yes, as associated data
`gateway_id`	2 bytes	Identifies the gateway	Yes, as associated data
`gateway_seq`	8 bytes	Strictly increasing per gateway	Yes, as associated data
`timestamp`	8 bytes	Milliseconds since epoch	Yes, as associated data
`nonce`	12 bytes	Fresh random value per message	Not authenticated; an input to GCM
`ciphertext + tag`	27 + 16 bytes	AES-256-GCM output over the frame	Yes, by construction

Total 74 bytes for a 27-byte frame — 47 bytes overhead, as measured. The key is read from a file or environment variable, never in source. The centre rejects a bad tag, a repeated or lower sequence number, and a timestamp outside a 30-second window.

C.6 Experimental design

Controls held constant across experiments: same frame format and alphabet, same device identity and starting sequence, same machine, fixed random seed for attack frame selection, and a warmed AES backend. **The only variable that changes between paired runs in E4 and E6 is the validator state.**

#	Experiment	Independent variable	Dependent variable	Controls	Runs	Output
E1	Break time	Cipher and attack technique	Time to recover the key; whether recovery succeeds at all	Same plaintext frame set, machine, key-generation procedure	30 randomly keyed trials per cipher–technique pair, fixed in advance	CSV + bar chart
E2	Latency	Gateway stage and legacy cipher	Mean processing time	Warmed backend, single process, no network, same frame	2000 per stage	CSV + table + chart
E3	Overhead	Frame length (27 characters standard; 28 for the Playfair filler case)	Envelope size and absolute overhead, in bytes and as a ratio	Same envelope structure and header fields throughout	One per frame length	CSV + table
E4	Detection rate	**Validation OFF / ON**, attack type (tamper gross, tamper small, inject, replay legacy, replay AES, downgrade)	Share of malicious frames rejected; where each was rejected	Identical attack set and seed in both states	Same set run twice, once per state	CSV + table
E5	ECB against GCM	AES mode	Repeated ciphertext blocks across repetitive telemetry	Same plaintext sequence and key	One run per mode	CSV + figure
E6	False alarms	**Validation OFF / ON**, reading profile (steady, ramping, step change at a breaker event)	Share of legitimate readings rejected; which check fired	Same generated stream in both states	Same stream run twice	CSV + table

E4 and E6 are the core and share a design: identical inputs, twice, differing only in validator state. E6 exists because E4 alone would mislead — a validator catches more attacks simply by tightening limits, at the cost of rejecting genuine readings, and a breaker event produces exactly the step change a rate-of-change check suppresses.

Testing *(see full report, C.7)*: round trip for every cipher including edge cases; Affine key validation; Playfair doubled characters, odd lengths and filler handling; envelope integrity against altered ciphertext, header and tag; replay rejection; end-to-end identity; validator behaviour per check. Run with `pytest`.

Tools *(see full report, C.8)*: Python 3.11+, assumed permitted by the course (OI3); ciphers from scratch, since implementing them is the coursework; AES via the `cryptography` library, because hand-rolled AES would be the largest correctness risk; `socket` for genuine process separation; `pytest`; `matplotlib`. Reproducibility rests on pinned versions, a no-argument entry point, a fixed seed, and the rule that every reported figure comes from a committed CSV.

C.7 Work plan

The plan is stated in relative weeks from the Milestone 2 submission, because the remaining milestone dates had not been issued when this document was prepared. The sequence and its dependencies are fixed, so the schedule verdict in B.4 rests on an ordering rather than on nothing. Owners are assigned per phase at the start of each week.

Stage	Content	Depends on	Status	Schedule
Milestone 2	This proposal, feasibility prototype, run log, three figures	—	**Complete**	Week 0
Next	P1 and P2: remaining four ciphers, device profiles, three-process split, logger	Prototype	Not started	Weeks 1–2
After	P3 and P4: remaining attacks, validator, systematic paired runs	P2 complete	Not started	Weeks 3–4
Final	P5: E1 and E3–E6, charts, report, demo rehearsal	P4 complete	Not started	Weeks 5–6

The six classical ciphers are mutually independent and individually testable, which makes Phase 1 the natural unit of division among three members. Phases 3 to 5 are strictly sequential, because each consumes the previous phase's output.

C.8 Threats to validity

Seven limitations are declared in advance *(full table: see full report, C.10)*: classical ciphers are not what legacy devices use (L1); timings mix pure Python with a compiled AES-NI library (L2); measurements come from one laptop with no network (L3); the validator's ranges are chosen by the team (L4); frames are short and structured, so classical cryptanalysis may underperform (L5); and the simulation omits real protocol framing (L6). Each is handled by declaring it where the affected result is reported, not by engineering around it.

L7 is the most serious: the attack set is written by the same team that writes the defence, risking a test of only what the validator already catches. The prototype shows the intended discipline — the small in-range edit was included precisely because it defeats the mitigation, and its success is reported rather than omitted.

Part D — Relevance of algorithms and techniques

D.1 Selection criteria

Algorithms were chosen against four criteria in order: named by the problem statement; able to operate on the telemetry alphabet (a reading contains digits, so a 26-letter alphabet is unusable — hence 36 characters, mod 36); models something real about legacy protection; and correct by construction where correctness matters, which is why AES comes from a library and the classical ciphers are written from scratch.

Course concept	Role in this project	What it demonstrates
Caesar, Additive	Weakest legacy protection	Key space small enough to try every key
Affine	Slightly stronger monoalphabetic	Still a fixed substitution, broken by known plaintext
Playfair	Digram substitution	Hides single-letter frequencies, not digram patterns
Vigenère	Polyalphabetic	Broken once the key length is found
Rail Fence	Rearrangement only	Content unchanged, so character counts leak
AES	Modern target	Confidentiality, and with GCM also integrity

D.2 Classical ciphers as a graded model

All operate on $x \in Z_{36}$, the index of a character in `A–Z` followed by `0–9`.

Cipher	Formula over Z_{36}	Key space (36 chars)	Legacy weakness modelled	Attack that breaks it	Supports
Caesar	$x + 3$	1	Obfuscation with no secret	Exhaustive search + recognition	V1, E1
Additive	$x + k$	36	Key too small to matter	Exhaustive search + recognition	V1, E1
Affine	$a \cdot x + b$, $\gcd(a,36)=1$	12 × 36 = 432	Fixed substitution dressed up	Known plaintext on two tags; exhaustive search	V1, E1
Playfair	Digram map on a 6 × 6 grid	$36!$ nominal, far less in practice	Hides letter frequency, not digram structure	Known plaintext on fixed tags	V1, E1
Vigenère	$x_i + k_{(i \bmod L)}$	36^L	Strongest legacy case studied	Kasiski / IoC for L, then per-position additive	V1, E1
Rail Fence	Permutation over r rails	≈ 25 usable values	Rearrangement without substitution	Exhaustive search; character counts leak	V1, E1

Read down the key-space column and the point appears: every value is either tiny or illusory. None is a defence; they are a measuring stick for how fast a defence fails. Playfair's nominal $36!$ is the clearest case — a keyword-derived grid occupies a tiny region of that space. (Per cipher rationale: see full report, D.2.)

D.2.1 Worked example

The same frame encrypted with each cipher. **All outputs produced by executed code**; Caesar and Vigenère match `docs/prototype_run.txt` exactly.

Cipher	Sample key	Ciphertext	Round trip
Caesar	shift 3 (fixed)	`G3AY5634L3485I7CCBE4T333456`	yes
Additive	k = 11	`OBI6DEBCTBCGDQFKKJMC1BBBCDE`	yes
Affine	a = 5, b = 8	`X43FEJ49M49TE7ODD8N9Q4449EJ`	yes
Playfair	keyword `GRID`	`I1D13Y12F6Z70J5449D3N33U123Y`	yes
Vigenère	keyword `GRIDKEY`	`JHFYC7O7Z84F63AQHBL5E6H84C7`	yes
Rail Fence	3 rails	`D2I29Q10V3105F9810027014B03`	yes

Two observations visible only in real output. The Playfair ciphertext is **28 characters, not 27**: the split is `D0 7V 23 01 I0 15 2F 49 98 B1 Q0 0X 01 23`, where the doubled `00` in the sequence field forces the reserved filler. The filler lands *inside* the frame, so the

parser must strip every `X`, not trailing ones — the reason K2 is in the risk register. The Rail Fence output contains exactly the plaintext characters reordered, which is its whole weakness.

D.3 AES-256-GCM as the migration target

Why AES. The published US federal standard for symmetric encryption, selected from the Rijndael family through open competition [15]: an open standard, maintained library implementations with hardware acceleration, and the algorithm already named by the commercial systems in Part B [11].

Why 256-bit. The measured cost is negligible — seal 3.15 µs, open 3.08 µs, against a pure-Python Caesar decrypt at 3.97 µs. Where the stronger option costs nothing measurable, take it.

Why GCM. SP 800-38D specifies GCM as "an algorithm for authenticated encryption with associated data" [16]. Confidentiality alone would not address Part A, where the hard half is authenticity and freshness rather than secrecy [3].

Mode	Confidentiality	Integrity	Pattern leakage	IV / nonce requirement
ECB	Weak in practice	None	**Severe** — identical plaintext blocks give identical ciphertext, and telemetry is highly repetitive	None, which is part of the problem
CBC	Yes	None by itself	Low	Unpredictable IV per message
CTR	Yes	None by itself	Low	Unique nonce per message
GCM	Yes	**Yes, tag over ciphertext and associated data**	Low	**Unique IV per key, 96 bits recommended** [16]

ECB appears in the project as a demonstration (E5), not a candidate. CBC and CTR are acceptable for confidentiality but give no integrity alone — precisely the gap that lets an altered envelope pass.

The 96-bit nonce. SP 800-38D recommends restricting IV support to 96 bits "to promote interoperability, efficiency, and simplicity of design" [16], and states the uniqueness bound: the probability of invoking authenticated encryption with the same IV and key on two distinct input sets "shall be no greater than 2^{-32}" [16]. Its caution is blunt — a breach "may compromise the security assurance almost entirely", GCM "should not be deployed unless compliance with this uniqueness requirement is ensured", and if even one IV repeats under a key the implementation "may be vulnerable to the forgery attacks" [16]. Hence a fresh random 96-bit nonce per message rather than a counter a restart could reset, and hence K3.

Associated data and tag. The header — version, gateway id, sequence, timestamp — is authenticated but not encrypted, which is correct: the centre must read the sequence and timestamp to reject replays before decrypting, yet must be certain they were not altered. Stage 4 confirms it: a flipped bit in the ciphertext and a flipped bit inside `gateway_id` were both rejected with `InvalidTag`. The contrast with Stage 3 is the point — GCM proves the envelope was not altered in transit; it cannot prove the measurement was true.

Alternatives *(see full report, D.4)*: ChaCha20-Poly1305 would work and is faster without AES-NI, excluded because the brief names AES rather than on merit; AES-CCM would work and is the stronger choice on a small microcontroller; AES-CBC + HMAC works if assembled correctly, rejected on safety of construction, not cryptographic grounds; TLS/DTLS would secure the uplink well but presumes a PKI, which is out of scope, and does nothing for the legacy link. The uplink is the easy part; this project's subject is the half none of them reaches.

D.4 Cryptanalytic techniques

Exhaustive search with a recognition test. Decrypt under every candidate key and recognise the right one structurally: does the candidate parse as a well-formed reading, with `D` at 0, `V` at 3, `I` at 8, `F` at 13, `B` at 18, `Q` at 20 and digits elsewhere? This is **ciphertext-only** — no matching plaintext is needed, only the public frame layout. All 36 additive keys fell in 0.2039 ms with exactly one candidate passing.

Known plaintext on the fixed tags. Each known pair gives `c ≡ a·p + b (mod 36)`; two pairs give a solvable system — *sometimes*. Using `D` (3) at position 0 and `I` (8) at position 8:

```
c_I − c_D ≡ a·(8 − 3) ≡ 5a (mod 36),   gcd(5,36) = 1,   5⁻¹ ≡ 29 (mod 36)
a ≡ 29·(c_I − c_D) (mod 36),  then  b ≡ c_D − 3a (mod 36)
```

For the worked example this gives `a ≡ 29 × 25 ≡ 5` and `b ≡ 23 − 15 ≡ 8`, recovering the key. **The failure case matters more.** Using `D` (3) at 0 and `V` (21) at 3, the difference is 18 and `gcd(18, 36) = 18`, which is not invertible: executed against the same

ciphertext, **all twelve** valid multipliers satisfy the system and nothing is recovered. The cause is that 36 is composite — a direct consequence of the alphabet chosen in D.1. An attacker must pick tag pairs whose index difference is coprime to 36.

Frequency analysis exploits uneven natural-language letter distribution, and a telemetry frame is not natural language: 27 characters, mostly near-uniform digits, six fixed tags. Its honest underperformance on short structured frames is itself a finding (Q1). **Kasiski and the index of coincidence** recover the Vigenère key length, after which the text splits into `L` additive ciphers; the same short-frame caution applies (E1).

D.5 Gateway validation techniques

Not cryptographic. They exist because cryptography cannot solve V2.

Check	What it catches	What it misses
Range	Values outside plausible limits — the 290.1 V edit was rejected as "V=2901 outside plausible range [2000, 2600]"	**Any edit inside the range.** The 231.1 V edit was accepted and sealed into a valid envelope
Rate of change	Implausible jumps between consecutive readings	Slow drift over many frames; and it risks rejecting genuine step changes at a breaker event (E6)
Sequence	Replayed or out-of-order frames, once the device counter is tracked	A replay that also rewrites the sequence field, since the legacy frame carries no authentication

The run log states it plainly: "Validation reduces the problem. It does not remove it." These are compensating controls in the sense of NIST SP 800-82 Rev. 3, which directs that where a legacy system cannot support a required control an alternative should be considered [1]. They narrow the attack surface; they do not close it.

D.6 Mapping of techniques to the project

Algorithm / technique	Problem it addresses	V#	O#	E#
Caesar, Additive	Model the weakest legacy protection	V1	O1, O3	E1
Affine	Model a fixed substitution with a larger key	V1	O1, O3	E1
Playfair	Model digram-level protection	V1	O1, O3	E1
Vigenère	Model polyalphabetic legacy protection	V1	O1, O3	E1
Rail Fence	Model transposition without substitution	V1	O1, O3	E1
AES-256-GCM	Confidentiality and integrity on the uplink	V5	O1, O2	E2, E3, E5
96-bit random nonce per message	Prevent catastrophic IV reuse [16]	V5	O1	argued from [16], no experiment
Associated data over the header	Authenticate sequence and timestamp while leaving them readable	V6	O1, O2	E4
Exhaustive search + recognition	Break small key spaces from ciphertext alone	V1	O3	E1
Known plaintext on fixed tags	Break Affine, Vigenère and Playfair using frame structure	V1	O3	E1
Frequency analysis	Test whether substitution survives on short frames	V1	O3	E1
Kasiski, index of coincidence	Recover Vigenère key length	V1	O3	E1
Tamper, inject, replay on the legacy link	Demonstrate authenticated false data and replay	V2, V6	O4	E4
ECB demonstration	Show that mode choice changes what leaks	V5	O4	E5
Downgrade to passthrough	Show protection collapsing to the legacy level	V7	O4	E4
Range, rate-of-change, sequence validation	Compensating control for V2 and V6	V2, V6	O5	E4, E6

V3, V4 and V8 are properties of the architecture rather than of any algorithm (A.5, C.2.1).

D.7 Honest limits

The classical ciphers are a teaching stand-in; every cipher-specific result is a statement about that cipher, not field practice, and the unprotected zero point is what the literature describes [4]. The attacks exploit a known frame layout, reasonable for a published protocol but not a general result on arbitrary data. Timings compare implementations, not algorithms. AES-GCM is used, not analysed: its security is taken as given from [15] and [16], and side-channel and fault attacks are out of scope. Deliberately excluded: TLS/DTLS, PKI and key distribution, post-quantum cryptography, and hardware security modules.

References

[1] K. Stouffer *et al.*, *Guide to Operational Technology (OT) Security*, NIST SP 800-82 Rev. 3, September 2023. doi: 10.6028/NIST.SP.800-82r3. https://csrc.nist.gov/pubs/sp/800/82/r3/final

[2] A. C-F. Chan and J. Zhou, "Toward Safe Integration of Legacy SCADA Systems in the Smart Grid," arXiv:2107.05863. https://ar5iv.arxiv.org/html/2107.05863

[3] P. P. Tsang and S. W. Smith, "YASIR: A Low-Latency, High-Integrity Security Retrofit for Legacy SCADA Systems," *Proc. IFIP TC 11 23rd Int. Information Security Conf.*, vol. 278, Springer, 2008, pp. 445–459. https://www.cs.dartmouth.edu/~sws/pubs/yasir-sec08.pdf

[4] S. Kaniewski, N. Lohmiller, M. Menth, and T. Heer, "Securing The Forgotten Link: Selecting and Evaluating Compensating Security Measures for Legacy Device Integration Paths," accepted for *Proc. ARES*, Linköping. https://atlas.cs.uni-tuebingen.de/~menth/papers/Menth26i.pdf

[5] CISA, *Cyber-Attack Against Ukrainian Critical Infrastructure*, Alert IR-ALERT-H-16-056-01, updated 20 July 2021. https://www.cisa.gov/news-events/ics-alerts/ir-alert-h-16-056-01

[6] IEC, *Power systems management and associated information exchange — Data and communications security — Part 1*, IEC TS 62351-1:2007. https://webstore.iec.ch/publication/6903

[7] IEEE, *IEEE Standard for Electric Power Systems Communications — Distributed Network Protocol (DNP3)*, IEEE Std 1815-2012. https://standards.ieee.org/ieee/1815/5414/

[8] IEEE Std 1711.1 (SSPP) and IEEE Std 1711.2-2019 (SSCP). Titles and designations as listed in NIST SP 800-82 Rev. 3, Appendix D [1].

[9] R. V. Solomakhin, P. P. Tsang, and S. W. Smith, "Predictive YASIR: High Security with Lower Latency in Legacy SCADA," Dartmouth College. https://cs.dartmouth.edu/~sws/pubs/sts10-draft.pdf

[10] U.S. DOE CESER, *Hallmark Cryptographic Serial Communication*, fact sheet. https://www.energy.gov/documents/hallmark-cryptographic-serial-communication-schweitzer-engineering-laboratories

[11] W. Jackson, "Encryption device planned for legacy SCADA installations," *Washington Technology*, January 2005. https://washingtontechnology.com/2005/01/encryption-device-planned-for-legacy-scada-installations/328769

[12] H.-D. Nguyen *et al.*, "Embedding the Real-Time AES-128 Encryption into Programmable Logic Controllers for Secure Modbus TCP/IP Communications in Industrial Control Systems," *Int. J. Integrated Engineering*, vol. 17, no. 9, pp. 45–57, 2025. https://journal.uthm.edu.my/index.php/ijie/article/view/22589

[13] D. Bhamare *et al.*, "Cybersecurity for Industrial Control Systems: A Survey," arXiv:2002.04124, 10 February 2020. https://arxiv.org/abs/2002.04124

[14] DNP Users Group, *Overview of DNP3 Protocol* and *Features of DNP3*. https://www.dnp.org/About/Overview-of-DNP3-Protocol

[15] NIST, *Advanced Encryption Standard (AES)*, FIPS 197, 26 November 2001, updated 9 May 2023. https://csrc.nist.gov/pubs/fips/197/final

[16] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, November 2007. https://csrc.nist.gov/pubs/sp/800/38/d/final

---

Open items and limitations

These are stated rather than concealed. Each is an evidential limit that affects how strongly a particular claim in this document can be read. Nothing here is unverified material presented as fact — where a source could not be retrieved, the dependent claim is

marked at the point it is made.

#	Limitation	Affects
OI1	The Modbus/TCP Security specification could not be retrieved: the published URL returns HTTP 404 and the specification index is JavaScript-rendered with no retrievable document link. The claim that Modbus carries no confidentiality or authentication therefore rests on secondary sources [1], [2], [3] rather than on the protocol specification	A.1, B.6
OI2	Stuxnet is not cited. No primary source was retrievable, and its mechanism — manipulation of PLC logic through engineering workstations — differs from the telemetry-interception problem studied here. It would be added only with a verified source and an explicit statement of relevance	A.6
OI3	Python 3.11 is assumed to be permitted by the course. The assumption is not confirmed in writing; if another language were required, the ciphers and envelope would need reimplementation, though the design and the experiments would be unaffected	C.6
OI4	The remaining milestone dates had not been issued when this document was prepared, so the work plan is stated in relative weeks from Milestone 2 rather than against a calendar	B.4, C.7
OI5	The titles and designations of IEEE Std 1711.1 and 1711.2-2019 are taken from the standards list in NIST SP 800-82 Rev. 3 Appendix D [1], not from the standards themselves. The IEEE pages are JavaScript-rendered and returned no readable content, and the standards are paywalled. No property of either standard beyond its title is claimed	A.6, B.2
OI6	Reference [4] states its venue as ARES, Linköping, while its file name suggests a 2026 publication year. The accepted-manuscript text is cited as it stands; the final year and volume are unconfirmed	[4]
OI7	The University of Messina proxy paper could not be opened (HTTP 403) and is cited nowhere. The proxy-based category in the review therefore rests on an analysis [4] rather than on an implementation	B.6
OI8	The specific guarantees of DNP3 Secure Authentication — in particular whether it provides confidentiality in addition to authentication — could not be confirmed from an openly readable source. The corresponding cells in the comparison table are marked rather than filled	B.2
OI9	Reference [9] is an author-hosted draft. Its content is cited as read; the final venue, volume and page numbers are unconfirmed	[9]
OI10	Both commercial entries are old: the Thales report dates from 2005 [11] and the Hallmark fact sheet is undated. No current vendor datasheet for a serial security gateway was obtainable, so the review establishes that the architecture is commercially established but says nothing about the present state of the market	B.1, B.6

These are distinct from the seven methodological threats to validity, L1 to L7, which are listed separately in C.8 and restated wherever an affected result is reported.